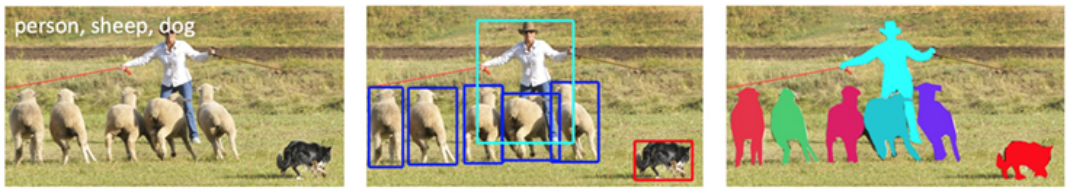
ELO328 - Procesamiento Digital de Imágenes

Clase 5: Segmentación de imágenes

Prof. Nicolás Torres
nicolas.torresr@usm.cl
Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

Introducción



(a) classification (b) detection (c) segmentation

Segmentación:

- Proceso de particionar una imagen digital en múltiples segmentos.
- Busca simplificar y/o cambiar la representación en algo que sea más fácil de analizar.
- Se utiliza para localizar objetos y regiones de interés.

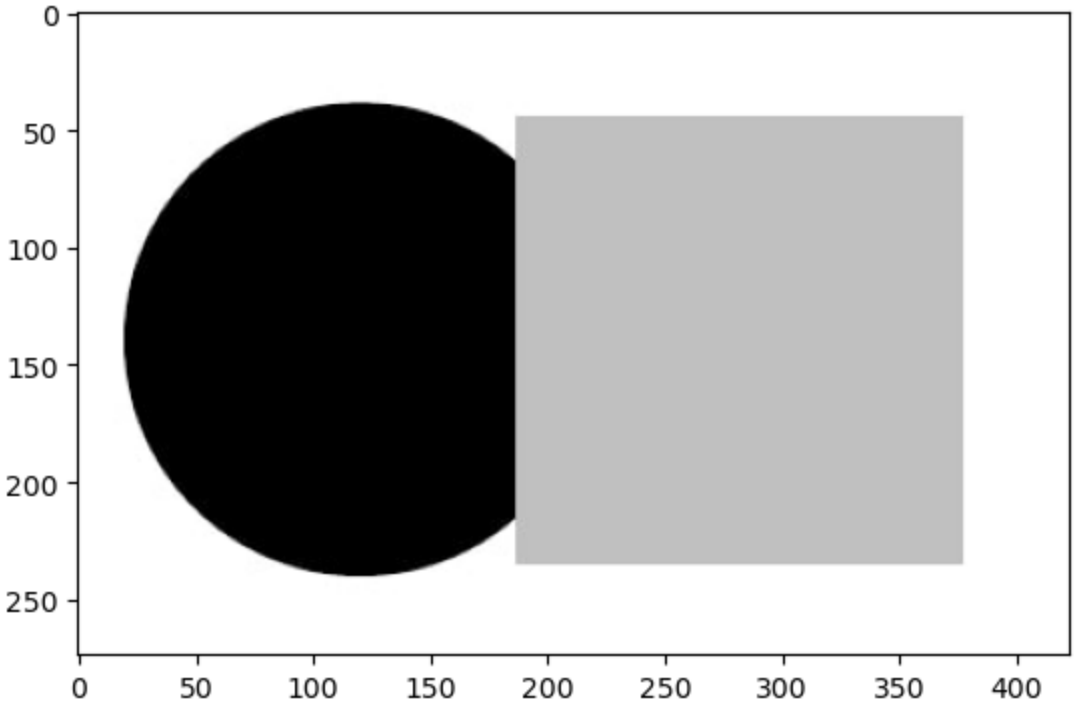
Objetivo: Identificar grupos de píxeles.

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
```

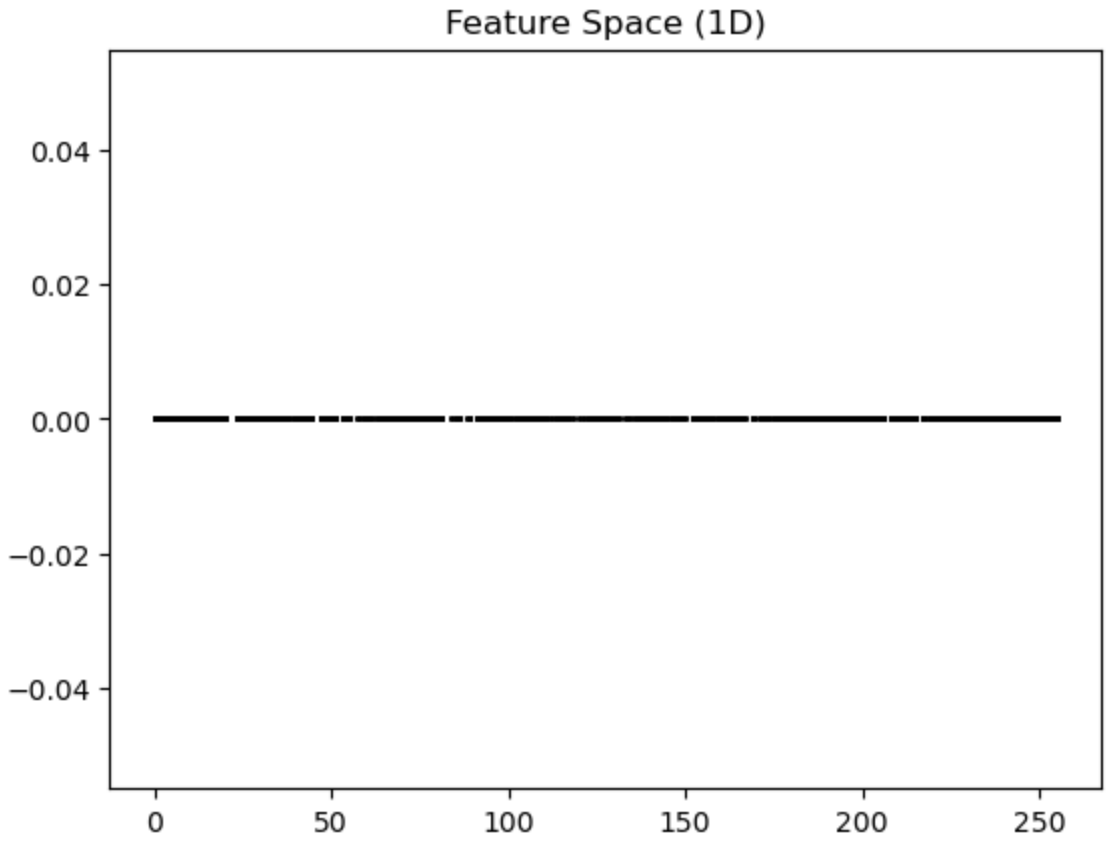
In [2]:
```
import warnings
warnings.filterwarnings("ignore")
```

In [3]:
```
from sklearn.cluster import KMeans
from skimage.color import rgb2gray
```

In [4]:
```
figuras = plt.imread("img/hist.jpg")
plt.imshow(figuras)
plt.show()
```



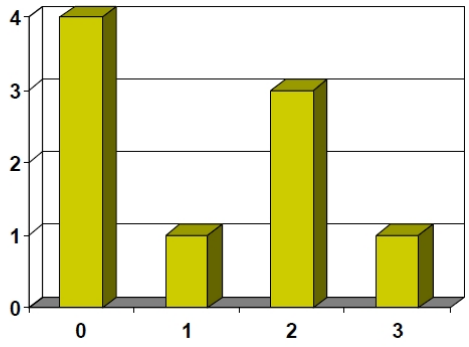
In [5]:
```
plt.scatter(figuras.flatten(),np.zeros(figuras.shape), c='black', s=1)
plt.title("Feature Space (1D)")
plt.show()
```



Histograma de intensidades

Histograma de una imagen: $h(k) = n_k$, con $k \in \{0, \ldots, L\}$ nivel de gris (para L niveles de gris), y n_k el número de píxeles de la imagen con valor de nivel de gris k.

0	2	2
3	0	2
0	0	1



- Es común también utilizar la versión normalizada del histograma en $(0, 1)$:

$$p(k) = \frac{n_k}{n}$$

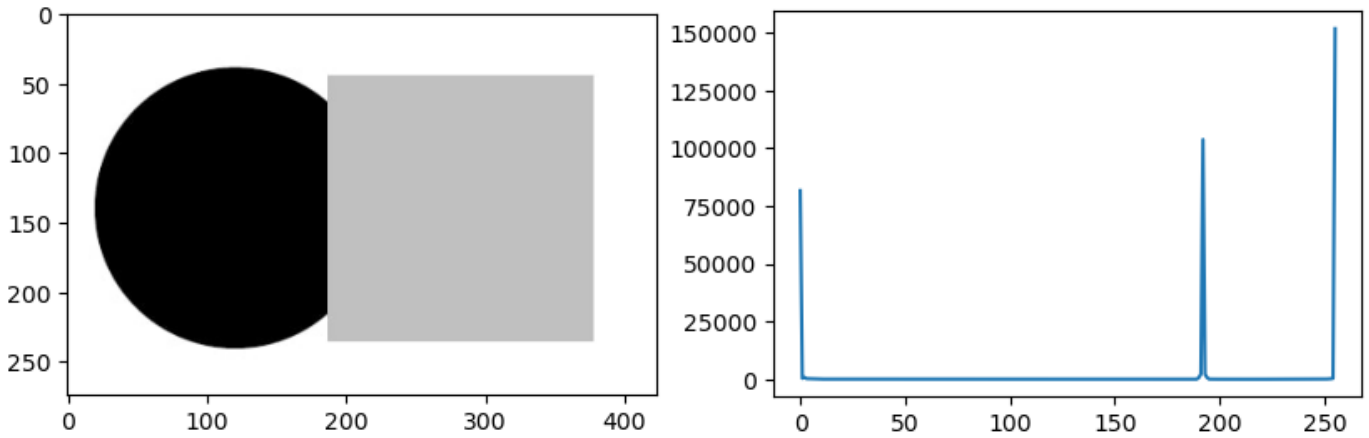
con n como el número total de píxeles en la imagen.

- Se puede decir que $p(k)$ es un estimador de la probabilidad de ocurrencia del nivel de gris k en la imagen.

- Histogramas son herramientas importantes para segmentación de imágenes en tiempo real (simples y baratos de calcular).

In [6]:
```
histogram, bin_edges = np.histogram(list(figuras.flatten()), bins=256)
```

In [7]:
```
plt.figure(figsize=(10, 3))
plt.subplot(1,2,1)
plt.imshow(figuras)
plt.subplot(1,2,2)
plt.plot(histogram)
plt.show()
```



Segmentation by thresholding

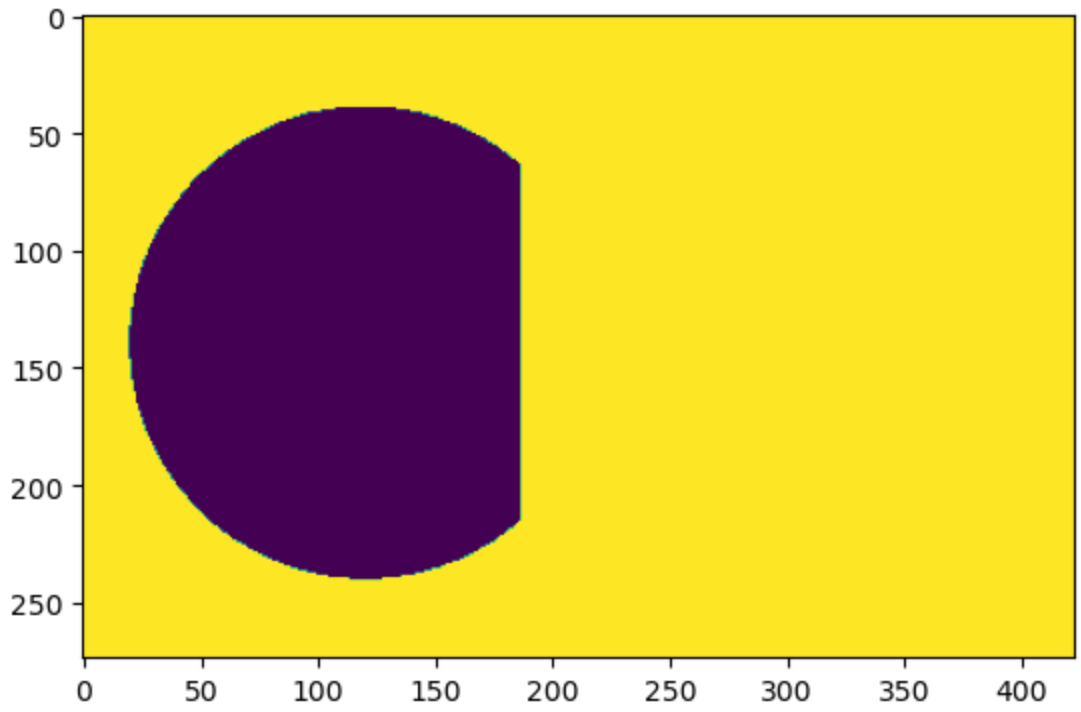
El método más simple de segmentación es la segmentación binaria por umbral.

Los píxeles con valor de intensidad menor a cierto umbral son 0 y los mayores o iguales al umbral son 1.

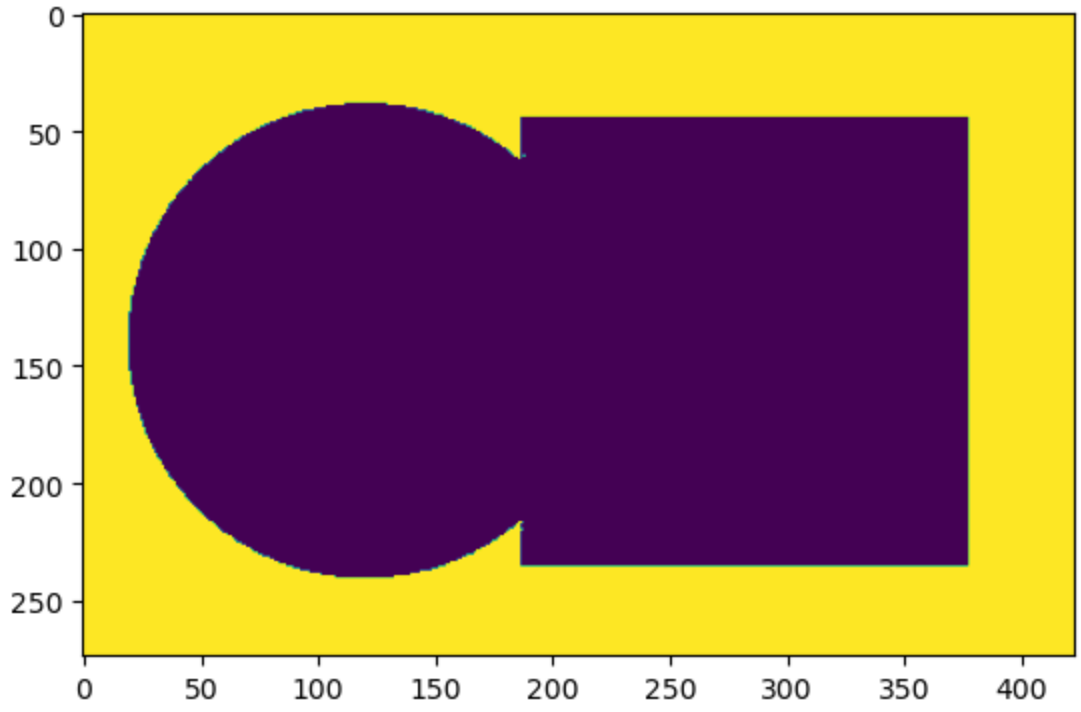
In [8]:
```
def plot_thresholding(image, t=0.5):
    binary=rgb2gray(image)
```

```
    binary[binary<t]=0
    binary[binary>=t]=1
    plt.imshow(binary)
    plt.show()
```

In [9]:
```
plot_thresholding(figuras, t=0.2)
```



In [10]:
```
plot_thresholding(figuras, t=0.8)
```

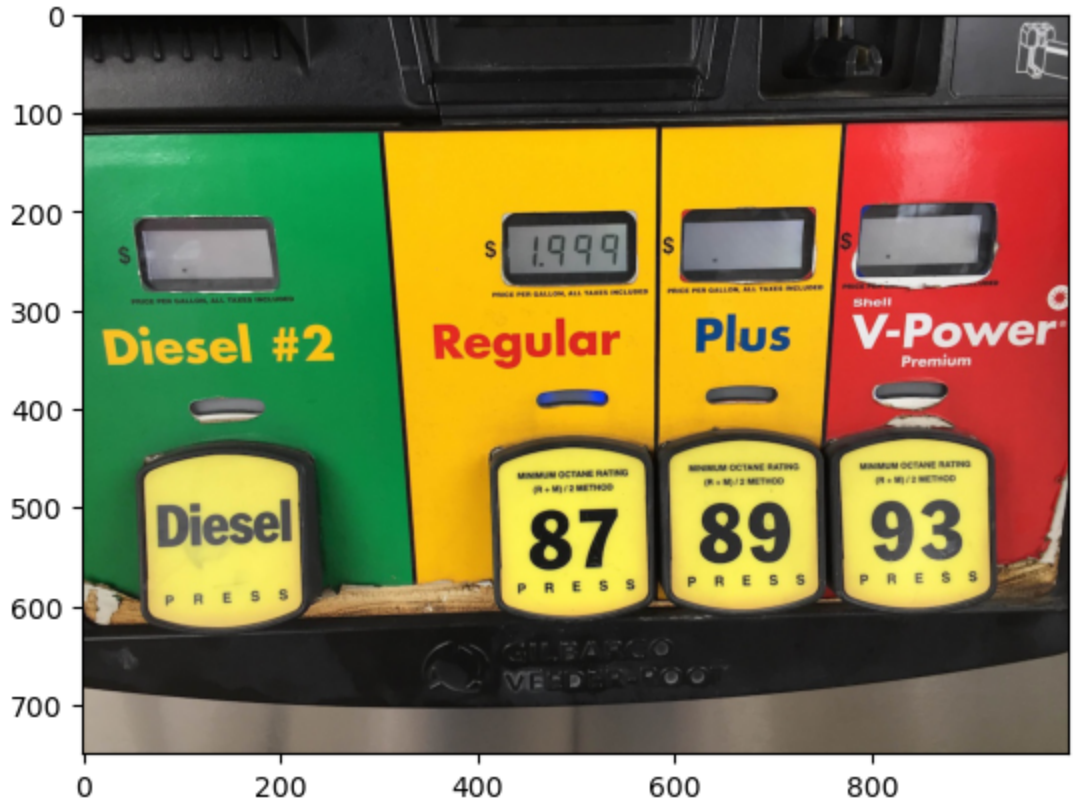


Agrupación de píxeles:

- 1D: Proximidad de la Intensidad.
- 3D: Proximidad de Color (RGB).

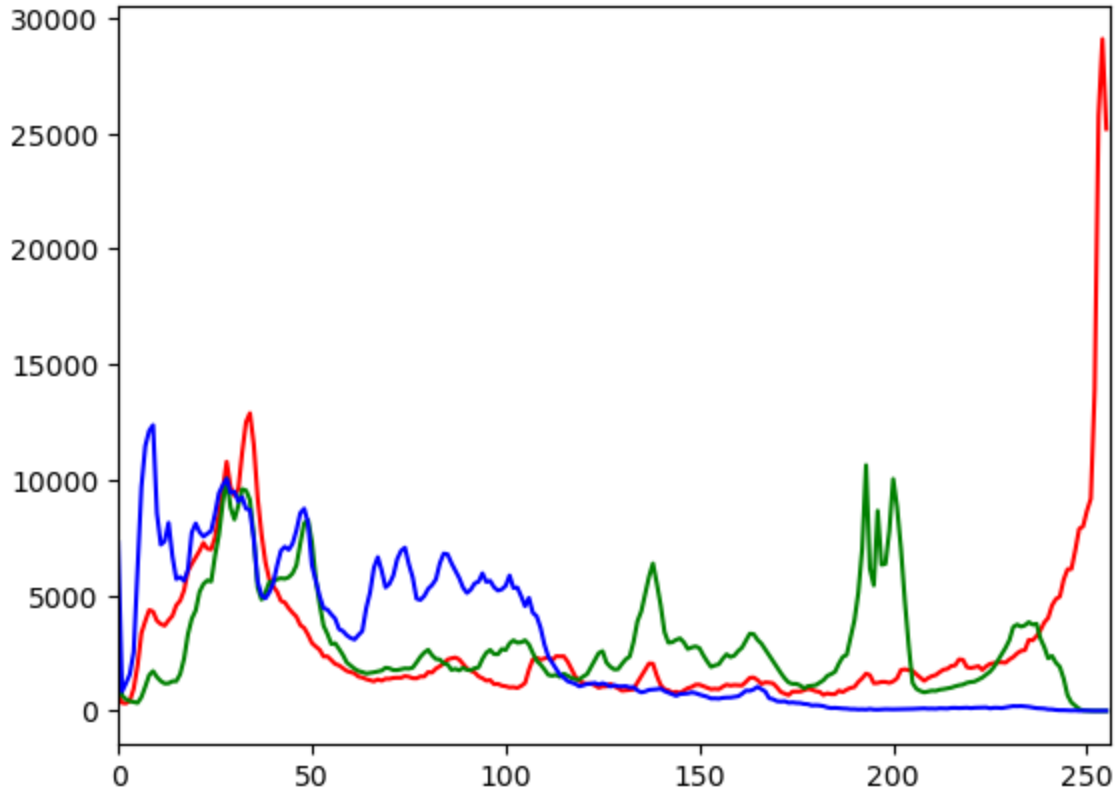
In [11]:
```
gas = plt.imread("img/gasoline.jpg")
```

```
plt.imshow(gas)
plt.show()
```



In [12]:
```
def plot_histogram_rgb(image):
    colors = ("red", "green", "blue")
    channel_ids = (0, 1, 2)
    plt.figure()
    plt.xlim([0, 256])
    for channel_id, c in zip(channel_ids, colors):
        histogram, bin_edges = np.histogram(image[:, :, channel_id], bins=256, range=(0, 256))
        plt.plot(bin_edges[0:-1], histogram, color=c)
    plt.show()
```

In [13]:
```
plot_histogram_rgb(gas)
```



In [14]:
```python
def plot_space_rgb(image):
    # Reshape the image to a 2D array of pixels
    pixels = image.reshape((-1, 3))

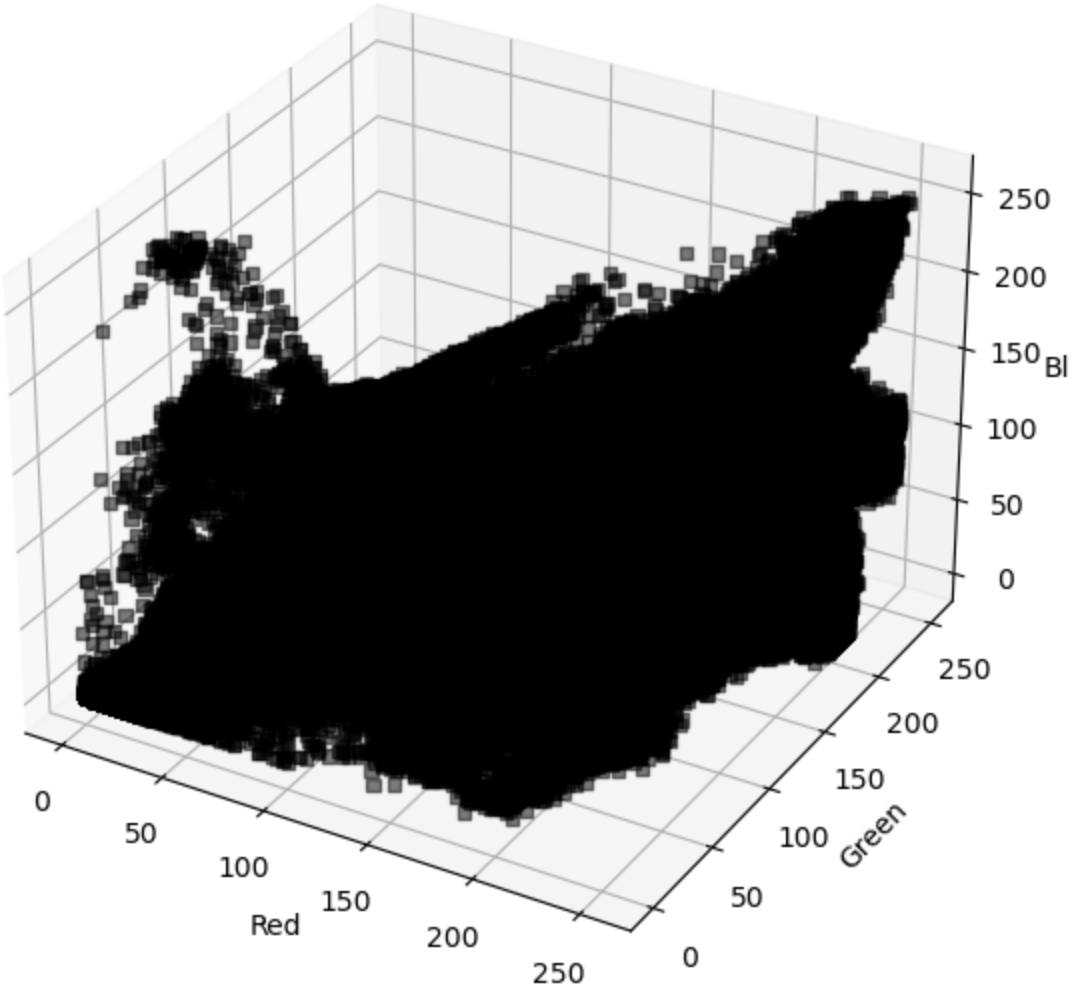
    # Extract individual color channels
    r = pixels[:, 0]
    g = pixels[:, 1]
    b = pixels[:, 2]

    # Create a 3D scatter plot
    fig = plt.figure(figsize=(10, 7))
    ax = fig.add_subplot(111, projection='3d')
    ax.scatter(r, g, b, marker='s', color='k', alpha=0.5)

    # Set labels for the axes
    ax.set_xlabel('Red')
    ax.set_ylabel('Green')
    ax.set_zlabel('Blue')

    plt.show()
```

In [15]:
```python
plot_space_rgb(gas)
```



K-Means

- Es una técnica de clustering que puede ser utilizada para segmentar una imagen en K grupos de píxeles con intensidad similar.
- El algoritmo asume la existencia de sólo K clusters, que agrupan píxeles según algún criterio de distancia.

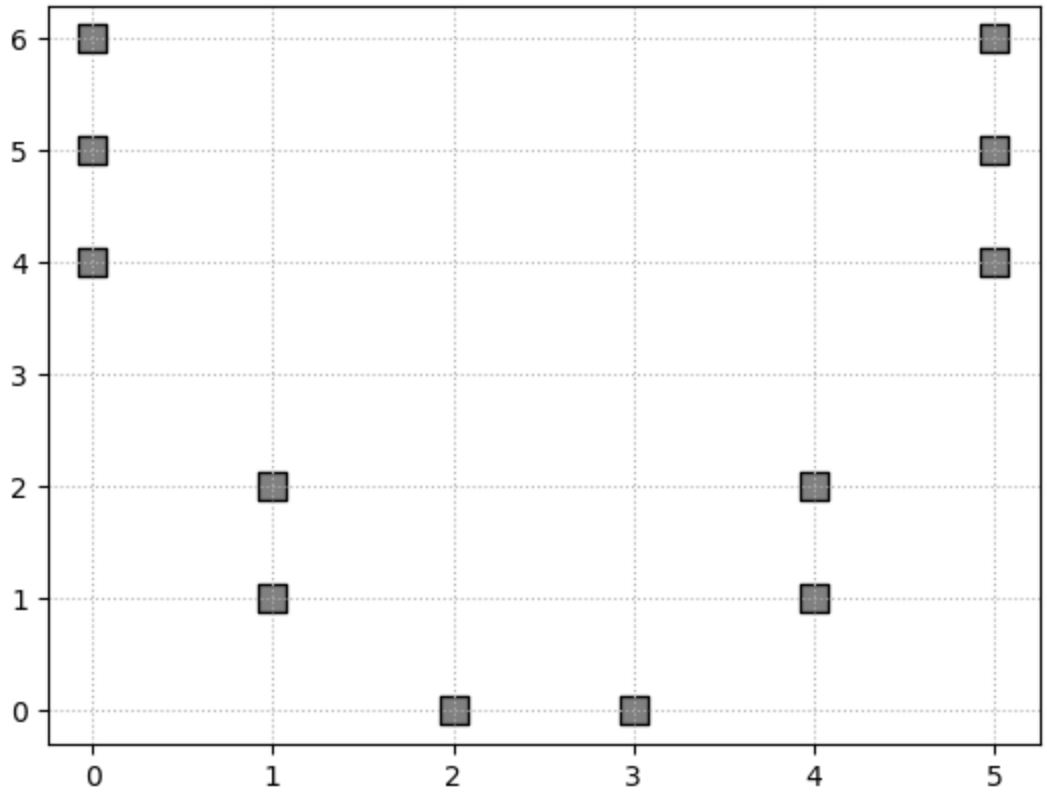
Algoritmo K-Means para segmentación de imágenes

1. Escoger K centroides de grupo, aleatoriamente o según algún otro criterio.
2. Asignar cada píxel de la imagen al centroide más cercano (menor distancia entre el píxel y el centroide).
3. Recalcular los centroides de los clusters promediando los píxeles de cada grupo.
4. Repetir 2 y 3 hasta converger (los píxeles no cambian de cluster entre iteraciones).

Ejemplo práctico K-Means en Python

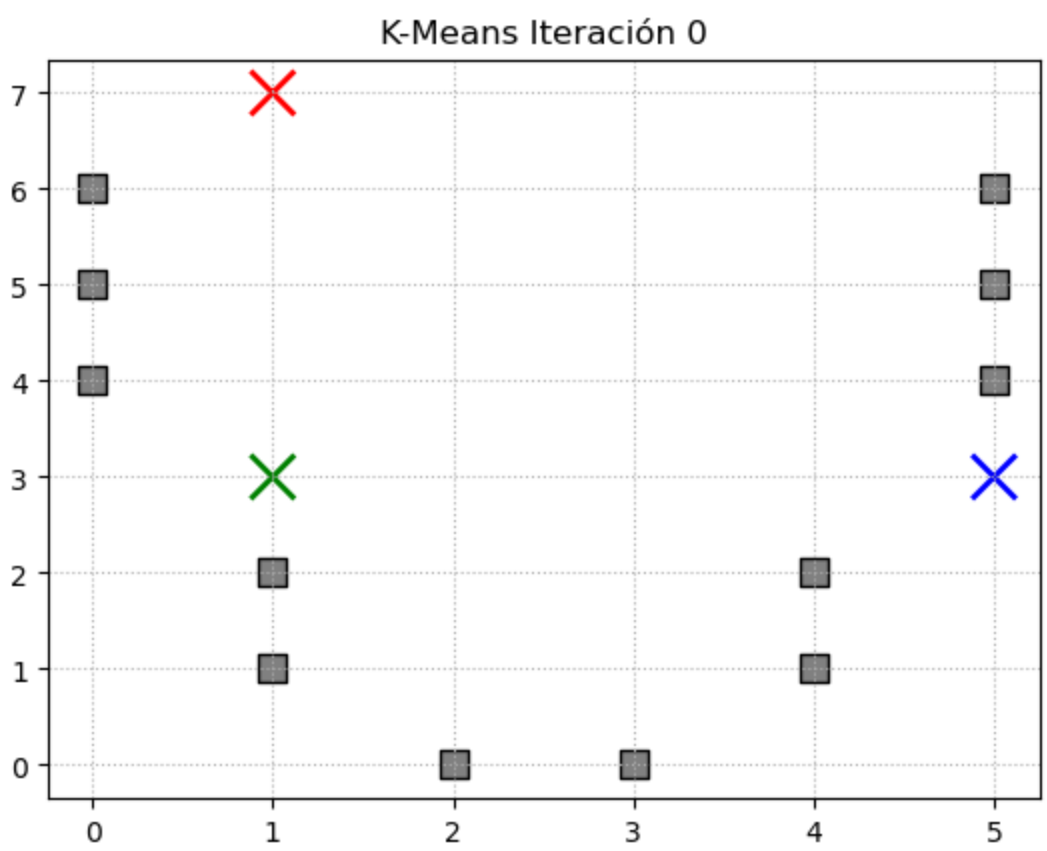
In [16]:
```
X = np.array([[0, 4], [0, 5], [0, 6],
```

```
                [1, 1], [1, 2],
                [2, 0], [3, 0],
                [4, 1], [4, 2],
                [5, 4], [5, 5], [5, 6],
               ])
plt.scatter(X[:, 0], X[:, 1], marker='s', c='gray', s=100, edgecolors='k')
plt.grid(linestyle="dotted")
plt.show()
```

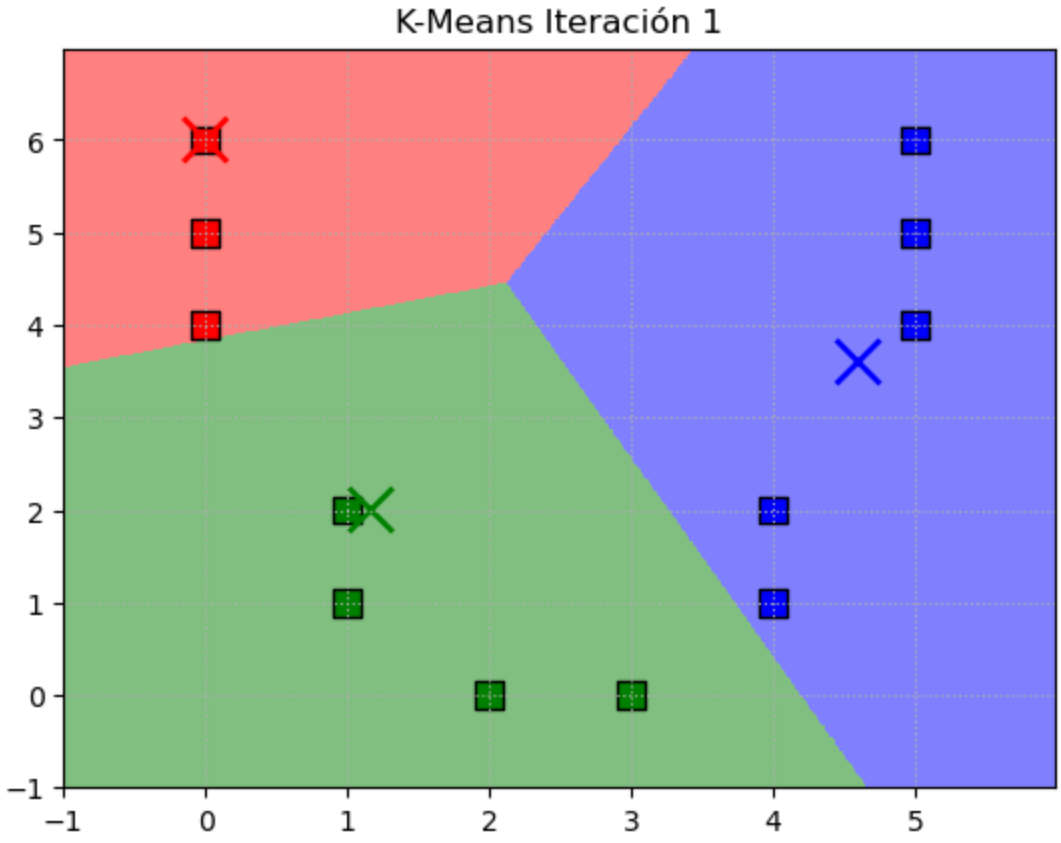


In [17]:
```
from plot_kmeans import plot_kmeans_example
```

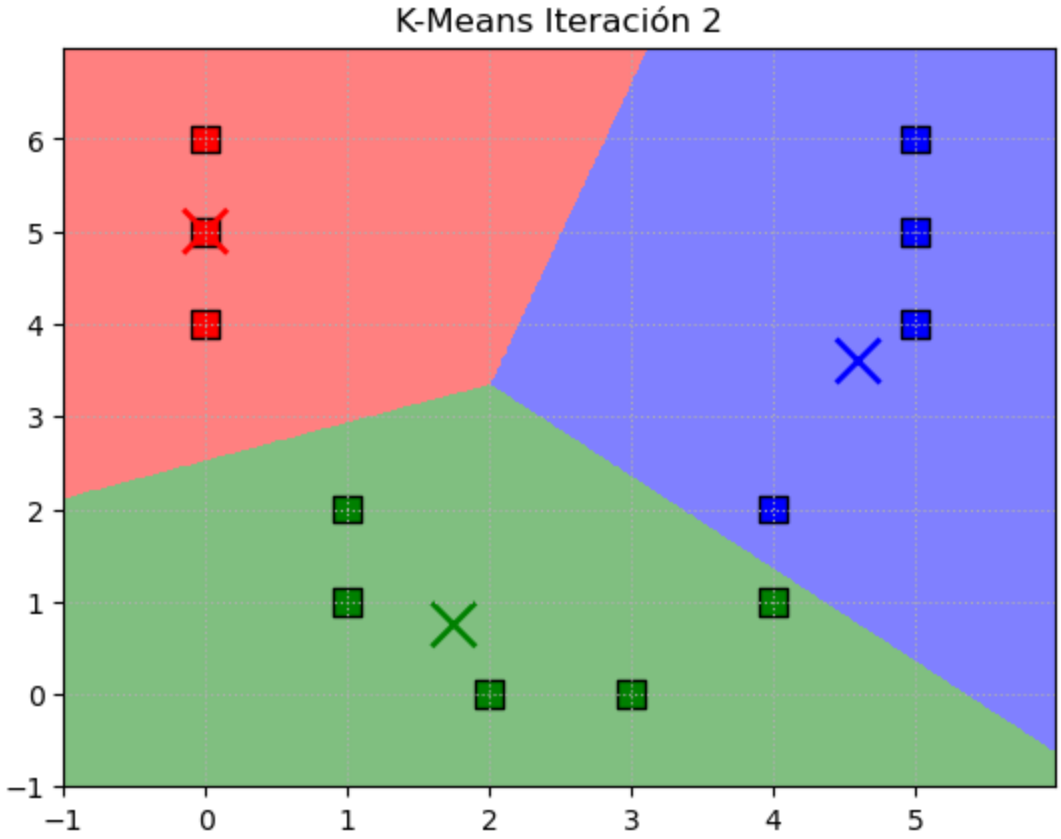
In [18]:
```
plot_kmeans_example(X, c1=(1,3), c2=(1,7), c3=(5,3), iterations=0)
```



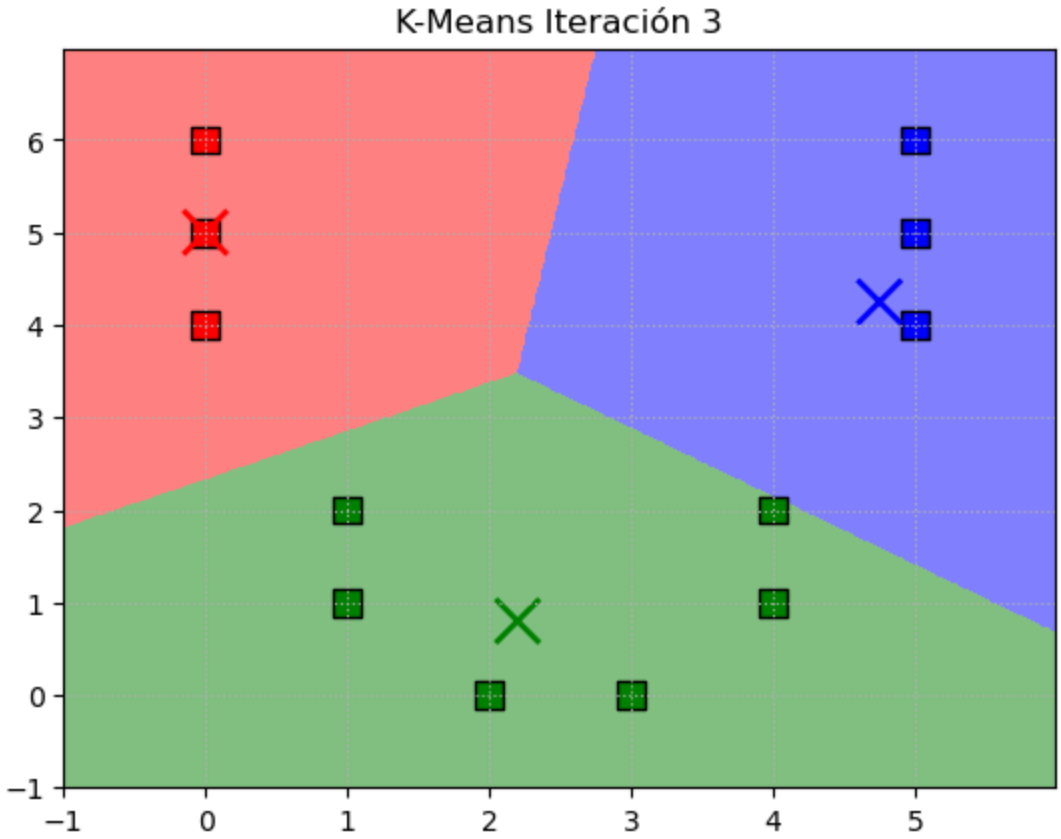
In [19]:
```
plot_kmeans_example(X, c1=(1,3), c2=(1,7), c3=(5,3), iterations=1)
```



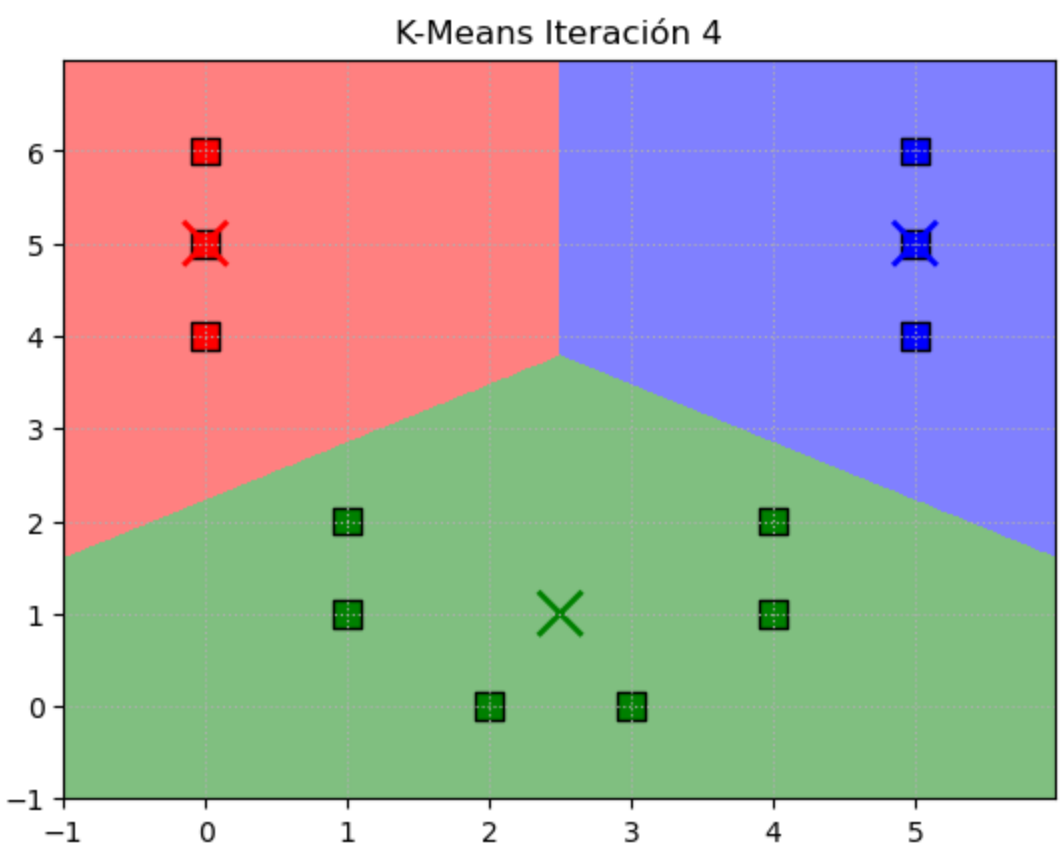
In [20]:
```
plot_kmeans_example(X, c1=(1,3), c2=(1,7), c3=(5,3), iterations=2)
```



In [21]:
```
plot_kmeans_example(X, c1=(1,3), c2=(1,7), c3=(5,3), iterations=3)
```



In [22]:
```
plot_kmeans_example(X, c1=(1,3), c2=(1,7), c3=(5,3), iterations=4)
```



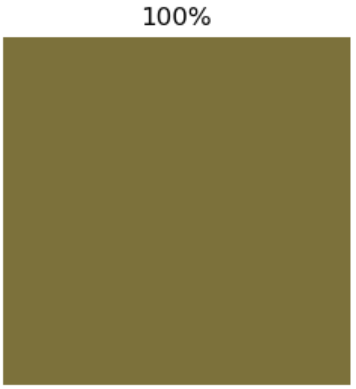
Color del cluster

Determinando colores dominantes en la imagen.

In [23]:
```
def dominant_colors(image, k=2):
    clustering = KMeans(n_clusters=k).fit(image.reshape(-1,3))
    i = 0
    plt.figure(0,figsize=(15,3))
    plt.subplot(1,k+1,1)
    plt.imshow(image)
    plt.axis('off')
    for mean_color in clustering.cluster_centers_:
        plt.subplot(1,k+1,i+2)
        plt.axis("off")
        color = np.zeros((100,100,3),dtype='uint8')
        color[:,:,:] = mean_color
        plt.imshow(color)
        percentage = list(clustering.labels_).count(i)/len(clustering.labels_)
        plt.title(f"{round(percentage*100)}%")
        i += 1
```

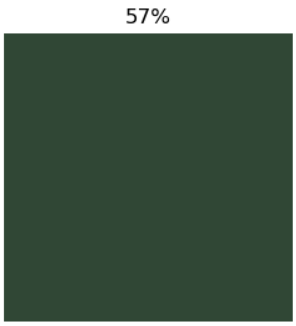
In [24]:
```
dominant_colors(gas, k=1)
```

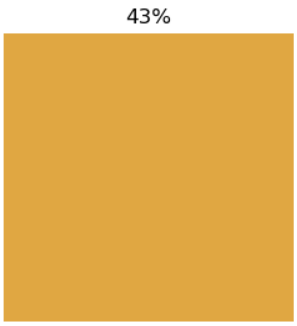




In [25]:
```
dominant_colors(gas, k=2)
```

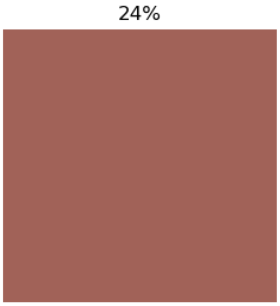


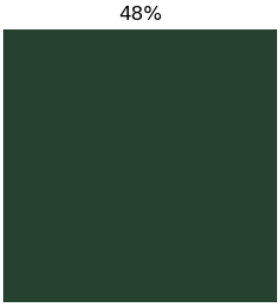


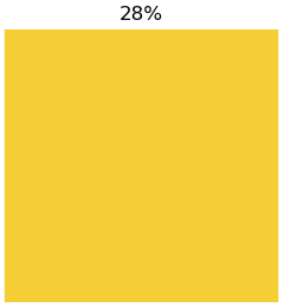


In [26]:
```
dominant_colors(gas, k=3)
```



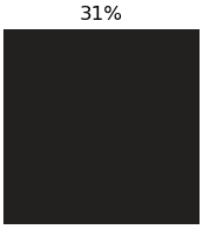


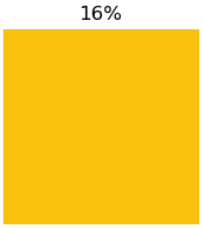


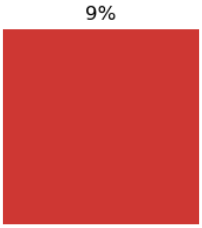


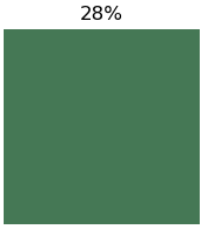
In [27]:
```
dominant_colors(gas, k=5)
```

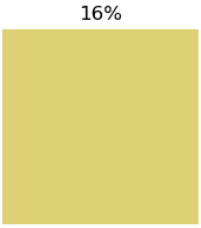










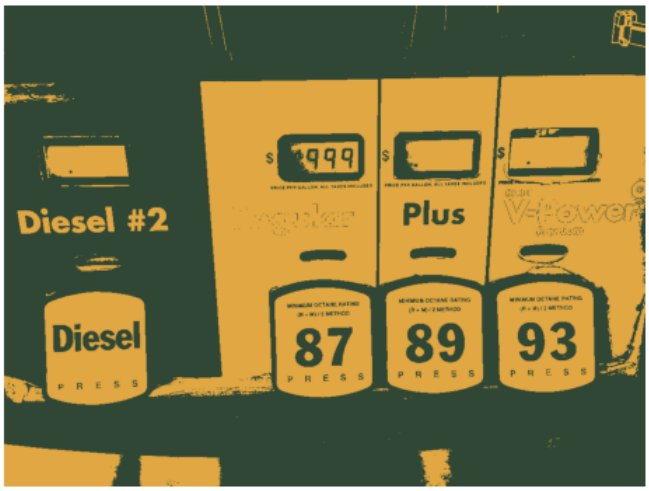


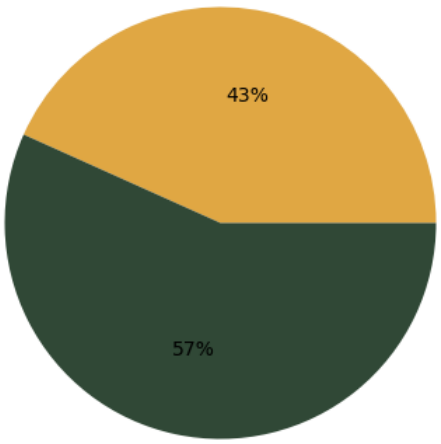
Segmentación con K-Means

In [28]:
```
def plot_segmentation_kmeans(image, k=2):
    clustering = KMeans(n_clusters=k).fit(image.reshape(-1,3))
    mean_colors = clustering.cluster_centers_.astype(np.uint8)
    segmented_image_mean_color = np.asarray([mean_colors[i] for i in clustering.labels_]).reshape
    list_of_percentages = []
    for i in range(len(clustering.cluster_centers_)):
        percentage = list(clustering.labels_).count(i)/(len(clustering.labels_))
        list_of_percentages.append(percentage)
    fig = plt.figure(figsize=(13, 5))
```

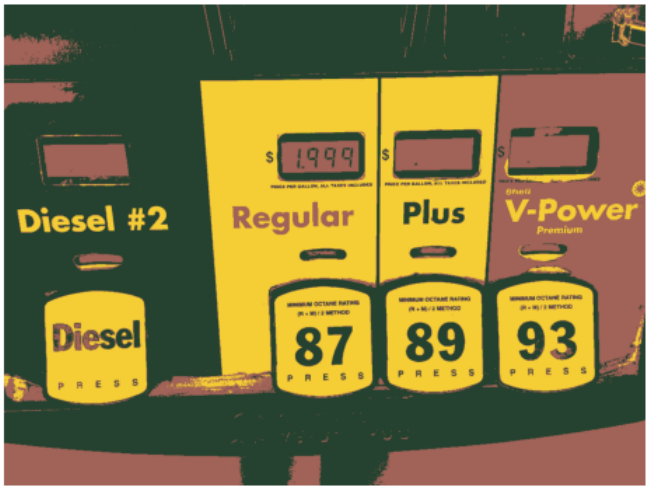
```
    ax1 = fig.add_subplot(121)
    ax2 = fig.add_subplot(122)
    ax1.imshow(segmented_image_mean_color)
    ax1.axis('off')
    ax2.pie(list_of_percentages,colors=np.array(clustering.cluster_centers_/255),autopct="%1.0f%%
```

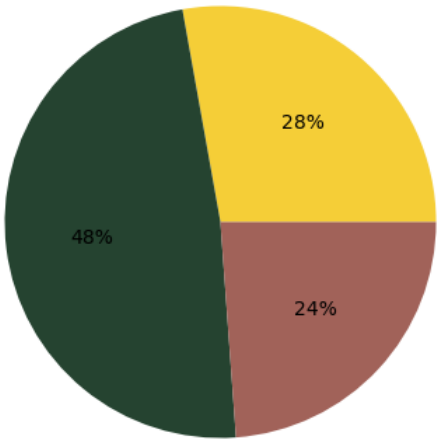
In [29]:
```
plot_segmentation_kmeans(gas, k=2)
```





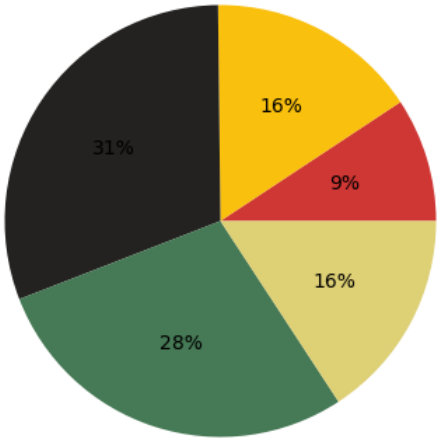
In [30]:
```
plot_segmentation_kmeans(gas, k=3)
```





In [31]:
```
plot_segmentation_kmeans(gas, k=5)
```





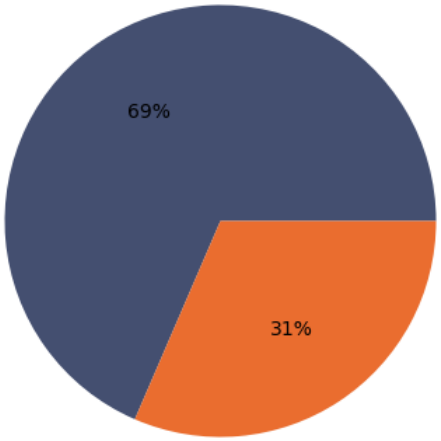
In [32]:
```
bird = plt.imread('img/bird.jpg')
```

In [33]:
```
plt.imshow(bird)
plt.show()
```

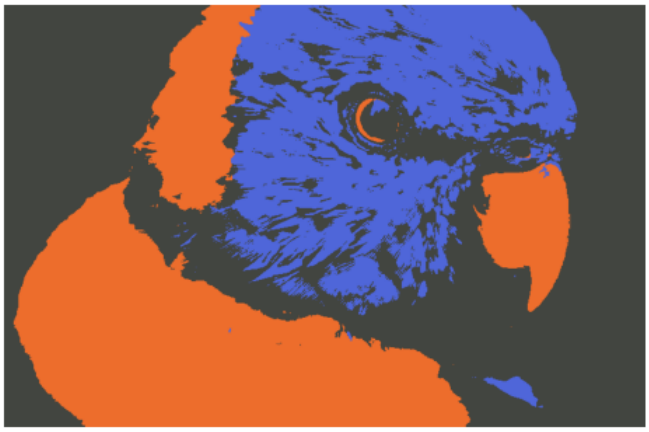


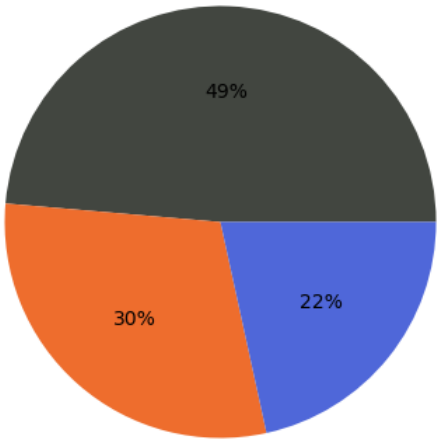
In [34]:
```
plot_segmentation_kmeans(bird, k=2)
```



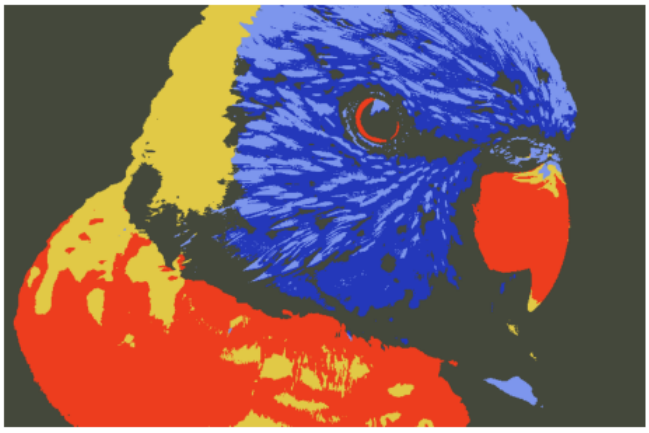


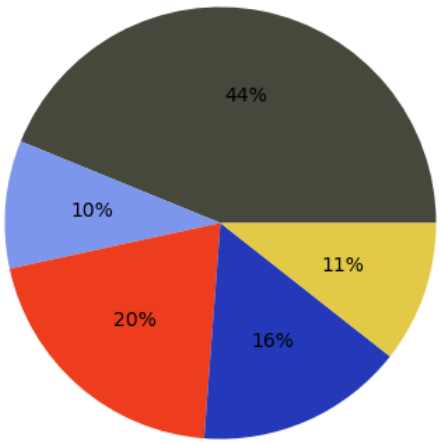
In [35]:
```
plot_segmentation_kmeans(bird, k=3)
```





In [36]:
```
plot_segmentation_kmeans(bird, k=5)
```





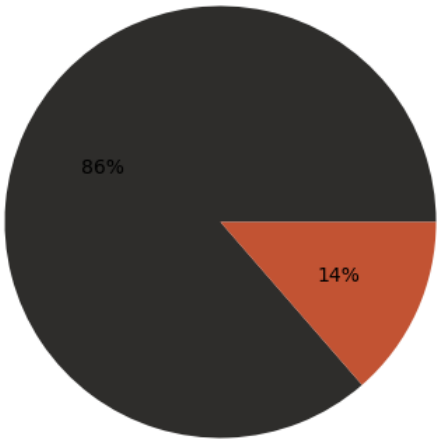
In [37]:
```
chili = plt.imread('img/chili.jpeg')
```

In [38]:
```
plt.imshow(chili)
plt.show()
```



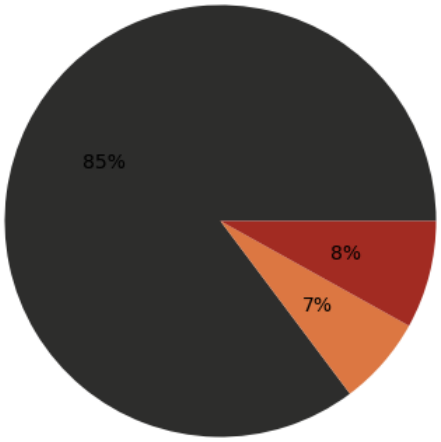
In [39]:
```
plot_segmentation_kmeans(chili, k=2)
```





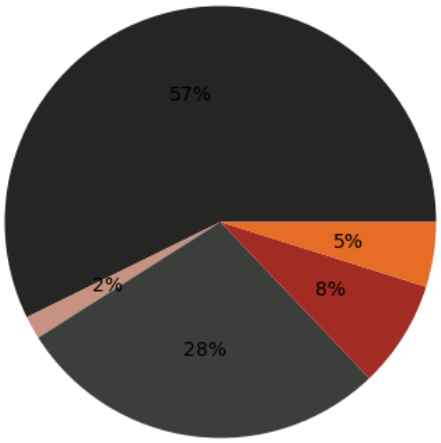
In [40]:
```
plot_segmentation_kmeans(chili, k=3)
```





In [41]:
```
plot_segmentation_kmeans(chili, k=5)
```





Segmentación basada en superpíxeles

Esta técnica se basa en usar clustering para extraer “superpixels” de una imagen.

¿Qué es un superpíxel?

Grupo de píxeles que comparten características comunes (como la intensidad).

Ventajas:

- Entregan más información que los píxeles individuales.
- Tienen un significado perceptivo ya que los píxeles que pertenecen a un superpíxel comparten propiedades visuales similares.
- Proporcionan una representación conveniente y compacta de imágenes que puede ser muy útil para problemas computacionalmente exigentes.

Simple Linear Iterative Clustering (SLIC)

- Modificación de K-Means para generar superpíxeles.
- SLIC asigna cada píxel a un vector de 5 dimensiones que contiene tres componentes de color y dos coordenadas espaciales.

$$\mathbf{z} = \begin{bmatrix} R \\ G \\ B \\ x \\ y \end{bmatrix}$$

donde (R, G, B) son las tres componentes de color del píxel y (x, y) son sus dos coordenadas espaciales.

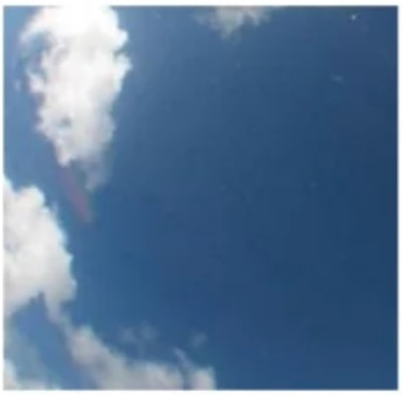
Principales etapas de SLIC

Sea N el número total de píxeles en la imagen y K el número deseado de superpíxeles,

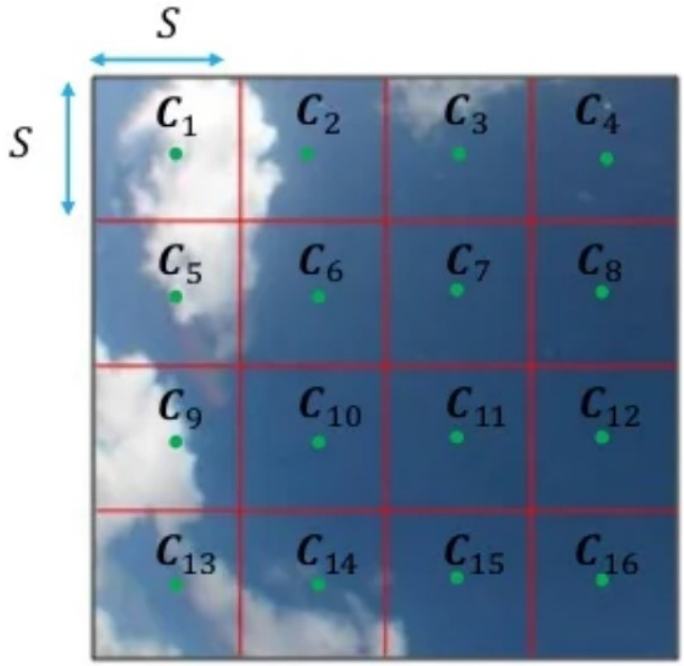
1. Inicializar K centroides $\mathbf{c}_k = [R_k G_k B_k x_k y_k]^T$ en una cuadrícula regular, donde cada celda tiene tamaño $S = \sqrt{\frac{N}{K}}$, y cada superpíxel tiene aproximadamente $\frac{N}{K}$ píxeles.

Por ejemplo,

- Image size: 40×40
- Number of pixels: $N = 1600$
- Number of superpixels: $K = 16$



Paso 1: Inicializar K centroides en la cuadrícula regular , donde cada celda tiene tamaño $S = \sqrt{\frac{N}{K}}$, y cada superpíxel tiene aproximadamente $\frac{N}{K}$ píxeles.

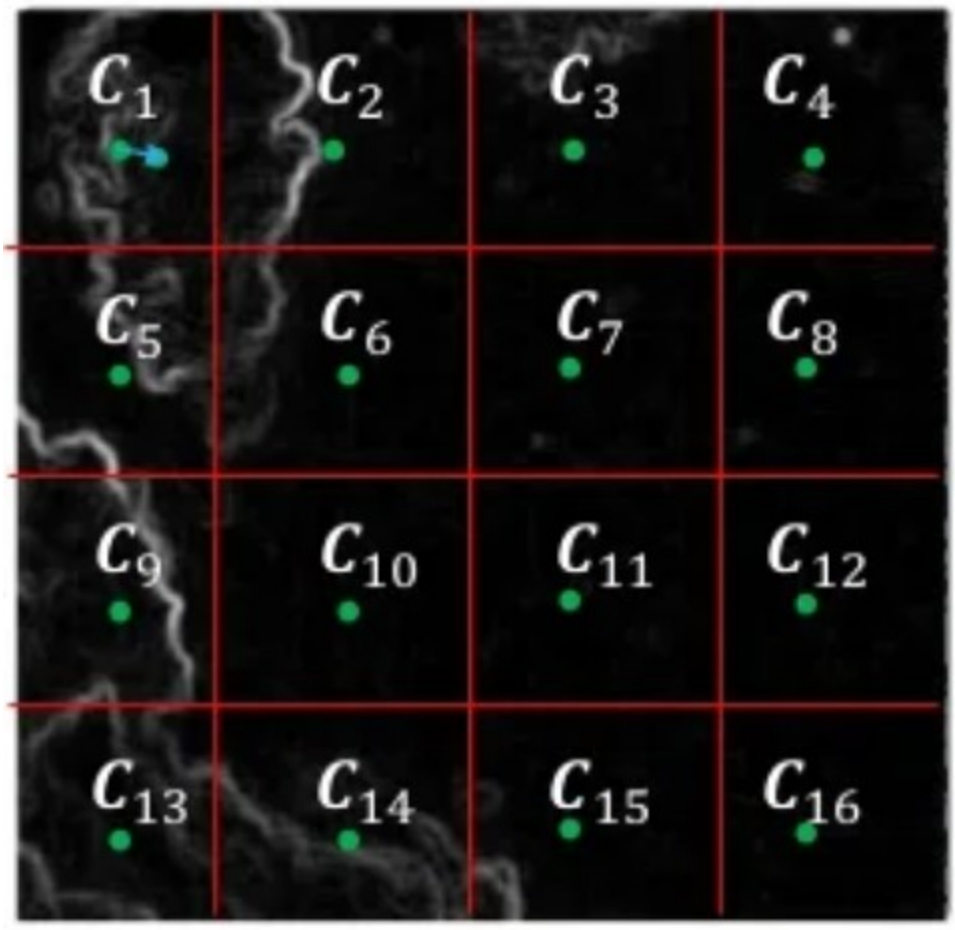


$$\frac{N}{K} = \frac{1600}{16} = 100$$

$$S = \sqrt{\frac{N}{K}} = \sqrt{100} = 10$$

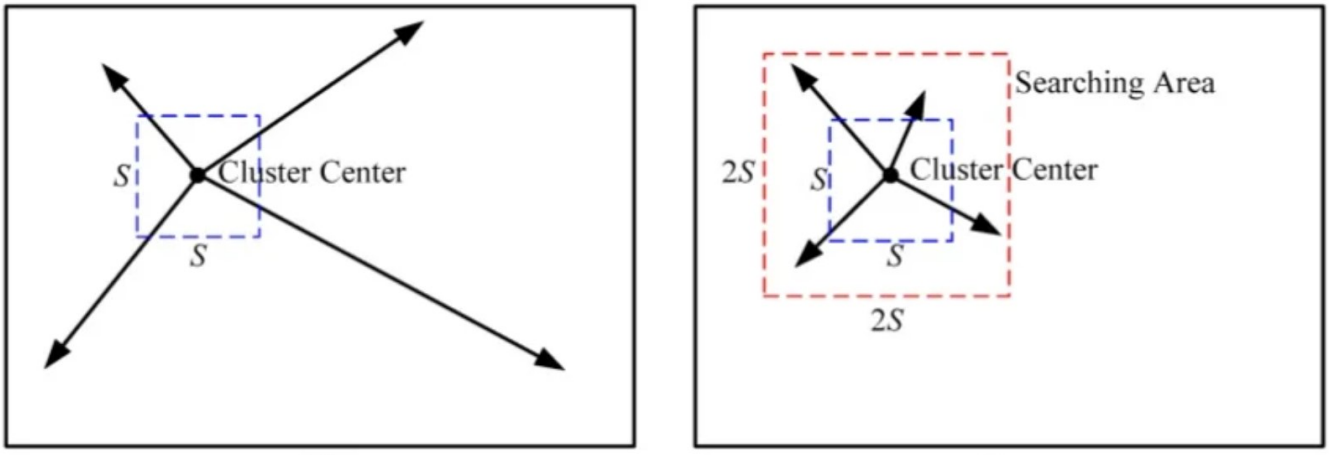
Principales etapas de SLIC

1. Mover los centroides iniciales a las posiciones con menor gradiente. Esto se hace para evitar centrar un superpíxel en un borde.



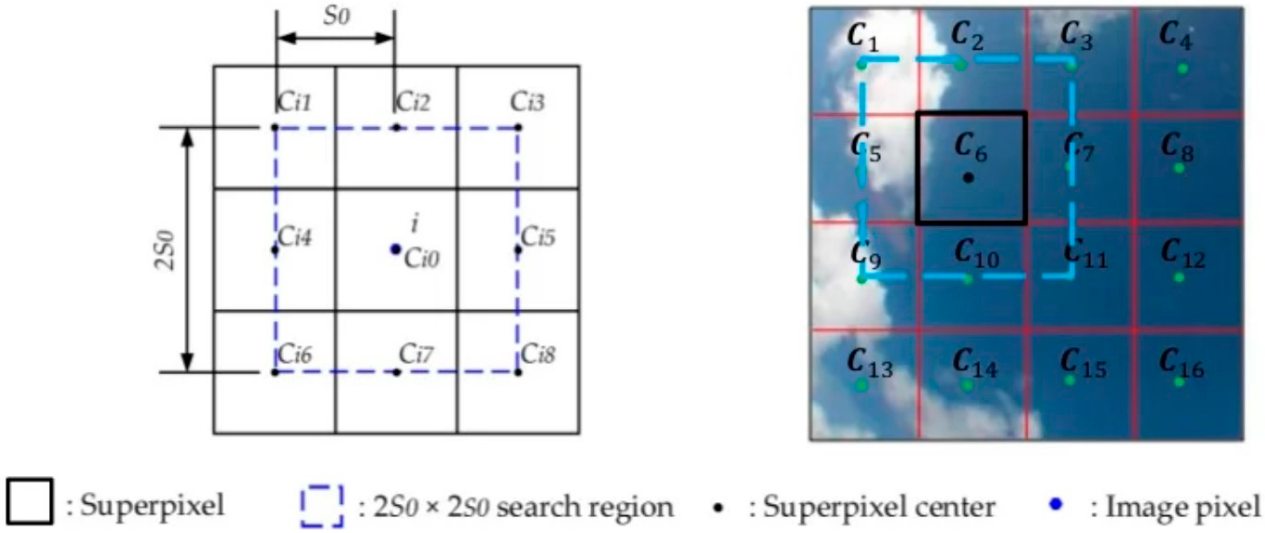
Principales etapas de SLIC

1. Asignar cada píxel a su centroide más cercano en un espacio de búsqueda local definido por un vecindario de $2S \times 2S$.



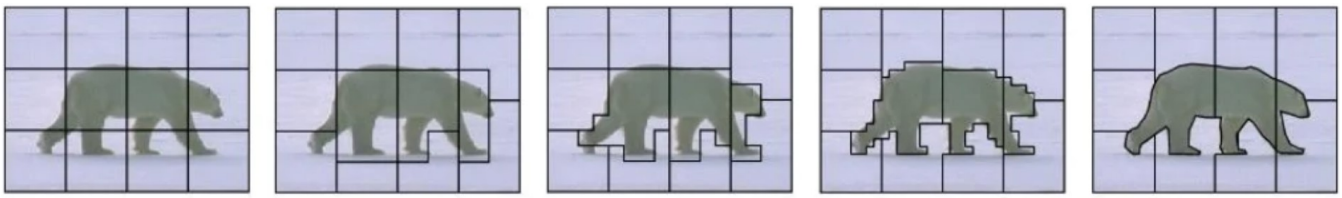
(a) K-means algorithm (b) SLIC algorithm

Siguiendo con el ejemplo, el **paso 3** sería:



Principales etapas de SLIC

1. Actualizar la ubicación de los centroides como el promedio entre todos los píxeles dentro del cluster correspondiente.
2. Repetir etapas **(3)** y **(4)** hasta lograr convergencia.



Ejemplo SLIC en Python

In [42]:
```
from skimage.segmentation import slic
```

In [43]:
```
def plot_slic(image, n_segments=3, sigma=0, compactness=10):
    segments = slic(image, n_segments=n_segments, sigma=sigma, start_label=0, compactness=compac

    # Calculate mean colors for each segment
    unique_labels = np.unique(segments)
    mean_colors = np.array([np.mean(image[segments == label], axis=0) for label in unique_labels

    # Assign mean colors to each pixel in the segmented image
    segmented_image_mean_color = mean_colors[segments]

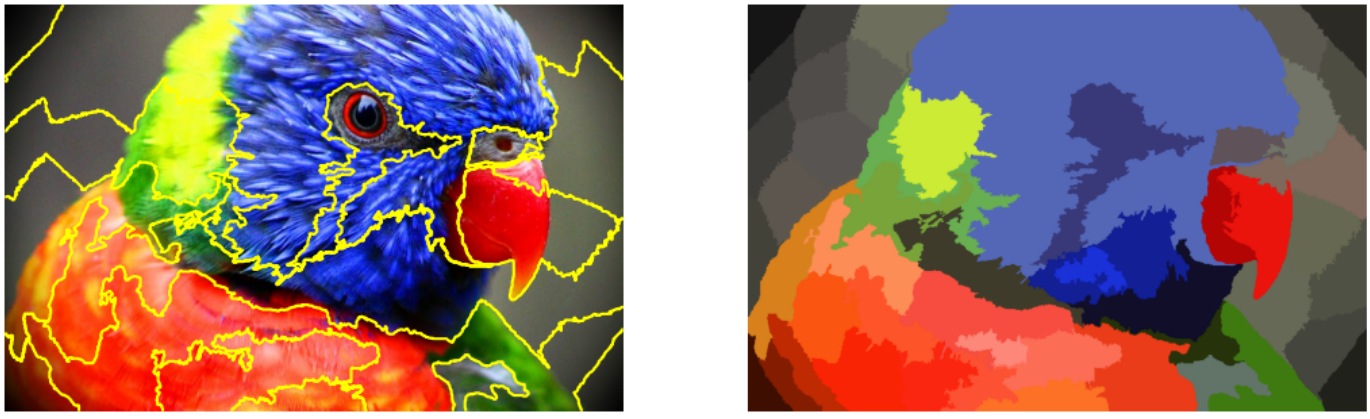
    fig = plt.figure(figsize=(13, 5))
    ax1 = fig.add_subplot(121)
    ax2 = fig.add_subplot(122)
    ax1.imshow(image)
    ax1.contour(segments, colors='yellow')
    ax1.axis('off')
    ax2.imshow(segmented_image_mean_color.astype(np.uint8))
    ax2.axis('off')

    plt.show()
```

In [44]:
```
plot_slic(bird, n_segments=5, sigma=0, compactness=10)
```

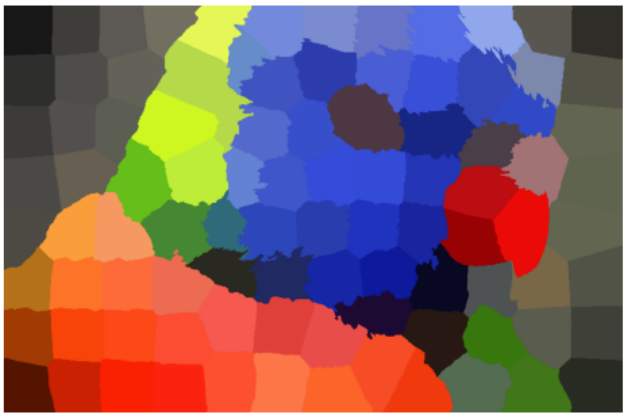


In [45]:
```
plot_slic(bird, n_segments=100, sigma=0, compactness=10)
```



In [46]:
```
plot_slic(bird, n_segments=100, sigma=0, compactness=100)
```





In [47]:
```
plot_slic(bird, n_segments=20, sigma=0, compactness=20)
```



